

Unit 2

Cloud Service and Deployments Models

(7 Hours)

. 2.1 Cloud service models:

. IaaS

. PaaS

. SaaS

. 2.2 Deployment models:

. Public cloud

. Private cloud

. Hybrid cloud

. Community cloud

. 2.3 Comparison of service models

. 2.4 Shared responsibility model

. 2.5 Selection criteria

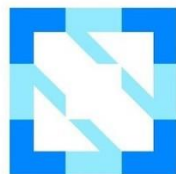
. 2.6 Multi-cloud concepts

. 2.7 Vendor lock-in issues

. 2.8 Serverless computing and event-driven architecture

Sanjeev Thapa, Er. DevOps, SRE, CKA, RHCSA, RHCE, RHCSA-Openstack, MTCNA, MTCTCE, UBSRS, HEv6, Research Evangelist

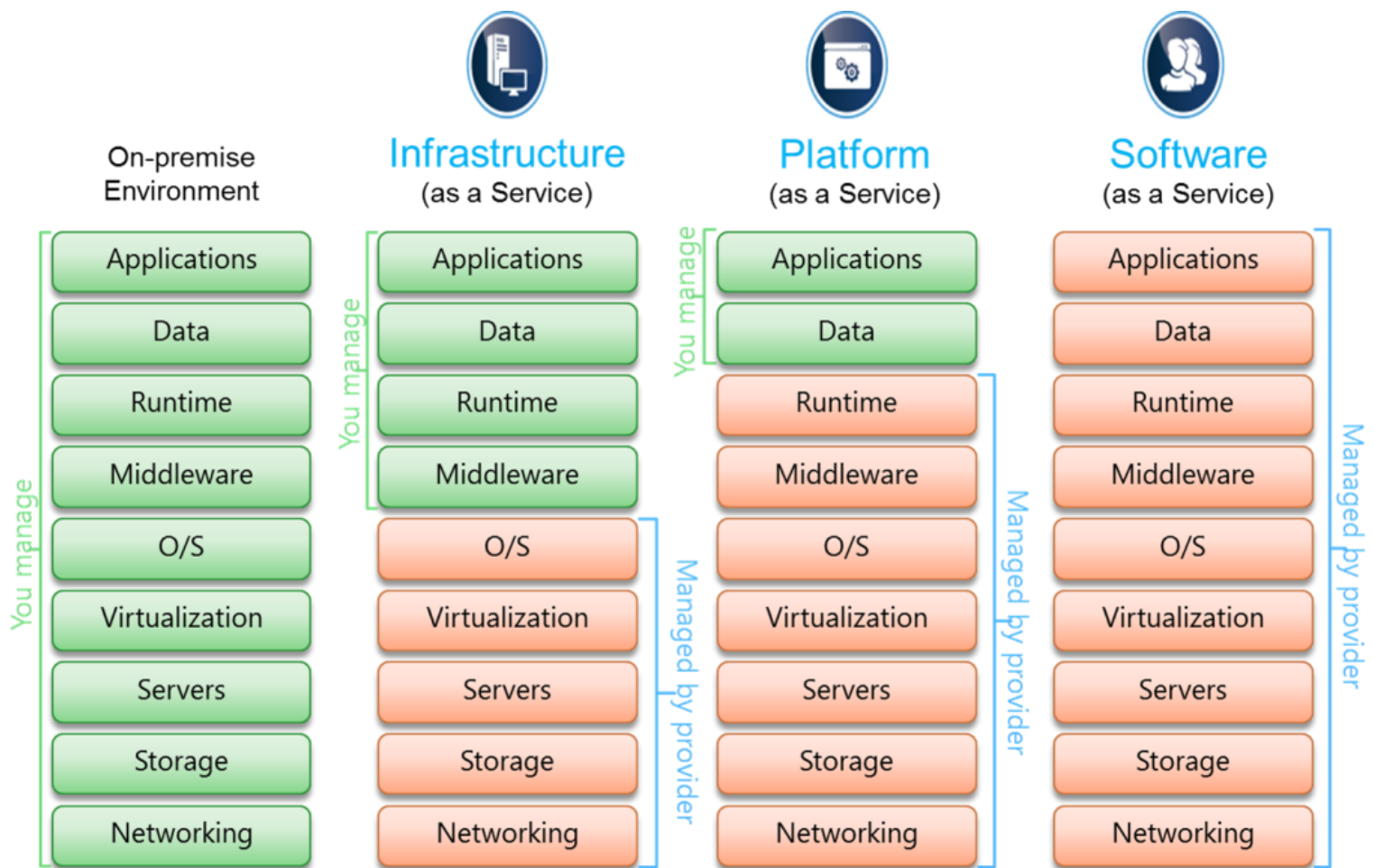
What is CNCF?



**CLOUD NATIVE
COMPUTING FOUNDATION**

What does it do?

. 2.1 Cloud service models:



Cloud service models explain **what type of cloud service is provided to users.**

The three major cloud service models are:

- . IaaS: Infrastructure as a Service
- . PaaS: Platform as a Service
- . SaaS: Software as a Service

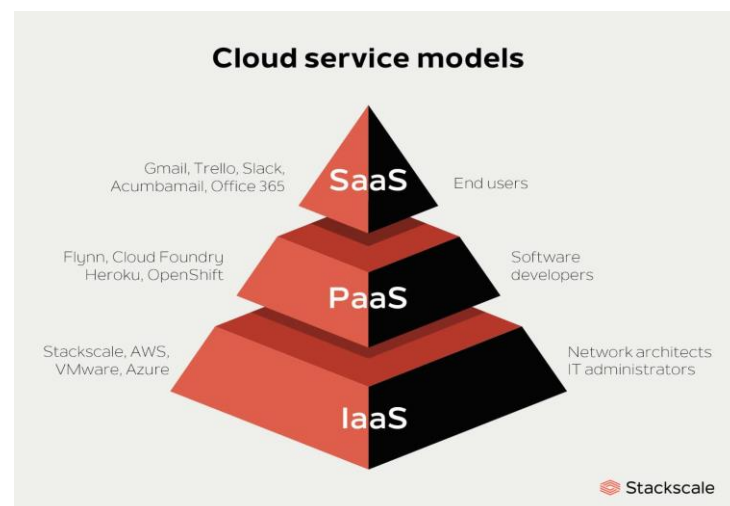
- . Virtual machines
- . Storage
- . Network
- . Firewall
- . Load balancer
- . IP address

A. IaaS: Infrastructure as a Service

Meaning

IaaS provides basic computing infrastructure through the internet.

In IaaS, cloud provider gives:



The user manages the operating system, applications, database, and security configuration.

Simple Example

A company needs a server to host its website.

Instead of buying a physical server, the company rents a virtual machine from a cloud provider.

Examples:

- . AWS EC2
- . Microsoft Azure Virtual Machine
- . Google Compute Engine
- . DigitalOcean Droplet
- . Linode VM

A startup needs 3 servers:

Server	CPU	RAM	Storage	Purpose
VM-1	2 cores	4 GB	80 GB	Web server
VM-2	4 cores	8 GB	160 GB	Application server
VM-3	4 cores	16 GB	250 GB	Database server

Instead of buying 3 physical servers, the startup rents 3 cloud VMs.

IaaS Responsibility Example

Component	Managed By
Physical server	Cloud provider
Data center	Cloud provider
Network hardware	Cloud provider
Virtualization	Cloud provider
Operating system	Customer
Application	Customer
Data	Customer

IaaS Cost Example

Assume cloud VM cost:

Small VM = Rs. 5/hour

Medium VM = Rs. 10/hour

Large VM = Rs. 20/hour

Monthly usage = 720 hours.

VM Type	Quantity	Cost/Hour	Monthly Cost
Small VM	1	Rs. 5	Rs. 3,600
Medium VM	1	Rs. 10	Rs. 7,200
Large VM	1	Rs. 20	Rs. 14,400

Total monthly cost:

$$3,600 + 7,200 + 14,400 = \text{Rs. } 25,200$$

- . High control over server
- . Flexible resource allocation
- . Suitable for custom applications
- . Easy scaling
- . No need to buy physical hardware

A developer creates a Python web application. Instead of managing Linux server, Python installation, web server, and scaling manually, the developer deploys code to a PaaS platform.

Examples:

- . Google App Engine
- . Heroku
- . Azure App Service
- . AWS Elastic Beanstalk
- . Render
- . Railway

Disadvantages of IaaS

- . Customer must manage OS and security
- . Requires technical knowledge
- . Misconfiguration risk
- . Cost can increase if resources are not monitored

Best Use Cases of IaaS

- . Hosting custom web applications
- . Running databases
- . Disaster recovery
- . Testing and development servers
- . Migrating old applications to cloud

PaaS Responsibility Example

Component	Managed By
Data center	Cloud provider
Server hardware	Cloud provider
Operating system	Cloud provider
Runtime	Cloud provider
Application code	Customer
Application data	Customer

B. PaaS: Platform as a Service

Meaning

PaaS provides a ready-made platform for developers to build, test, deploy, and manage applications.

In PaaS, the cloud provider manages:

- . Server
- . Operating system
- . Runtime environment
- . Middleware
- . Database platform
- . Scaling platform

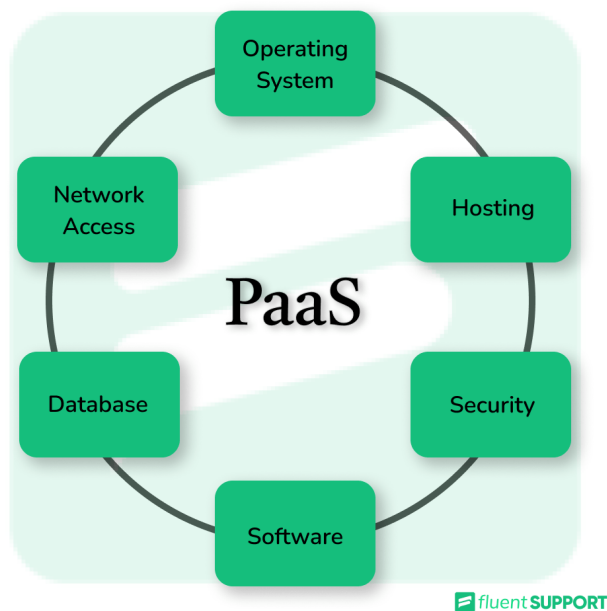
The developer mainly focuses on application code.

PaaS Example Scenario

A college wants to build an online admission form.

The developer writes:

- . Form page
- . Validation logic
- . Database connection
- . Email notification logic



The PaaS platform manages:

- . Server setup
- . Runtime
- . Scaling
- . Deployment
- . Basic monitoring

Advantages of PaaS

- . Faster application development
- . No need to manage server OS
- . Easy deployment
- . Built-in scaling features
- . Good for developers

Disadvantages of PaaS

- . Less control than IaaS
- . Platform dependency
- . Vendor lock-in risk
- . Limited customization

Best Use Cases of PaaS

- . Web application development
- . API development
- . Mobile backend
- . Rapid prototyping
- . Developer-friendly application hosting

C. SaaS: Software as a Service

Meaning

SaaS provides complete ready-to-use software through the internet.

The user does not manage server, operating system, database, or application code.

The user only uses the software.

Simple Example

A teacher uses Google Docs to prepare notes.

The teacher does not manage:

- . Server
- . Storage system
- . Software installation
- . Updates
- . Database

Google manages everything.

Examples:

- . Gmail
- . Google Docs
- . Microsoft 365
- . Zoom
- . Salesforce
- . Canva
- . Dropbox
- . Google Classroom

SaaS Responsibility Example

Component	Managed By
Data center	Cloud provider
Server	Cloud provider
Operating system	Cloud provider
Application	Cloud provider
Updates	Cloud provider
User data and access	Shared / Customer controls access

Need	SaaS Tool
Spreadsheet	Google Sheets

SaaS Licensing Data Example

A college has 80 staff members.

Cloud office software cost = Rs. 700 per user per month.

$$\text{Monthly Cost} = \text{Number of Users} \times \text{Cost per User}$$

$$\text{Monthly Cost} = 80 \times 700$$

$$\text{Monthly Cost} = \text{Rs. } 56,000$$

Yearly cost:

$$\text{Yearly Cost} = 56,000 \times 12$$

$$\text{Yearly Cost} = \text{Rs. } 672,000$$

SaaS Example in Education

A college uses Google Workspace.

Need	SaaS Tool
Email	Gmail
Online class	Google Meet
File storage	Google Drive
Notes	Google Docs
Assignment	Google Classroom

Advantages of SaaS

- . Easy to use
- . No installation required
- . Accessible from anywhere
- . Automatic updates
- . Low technical maintenance
- . Good for collaboration

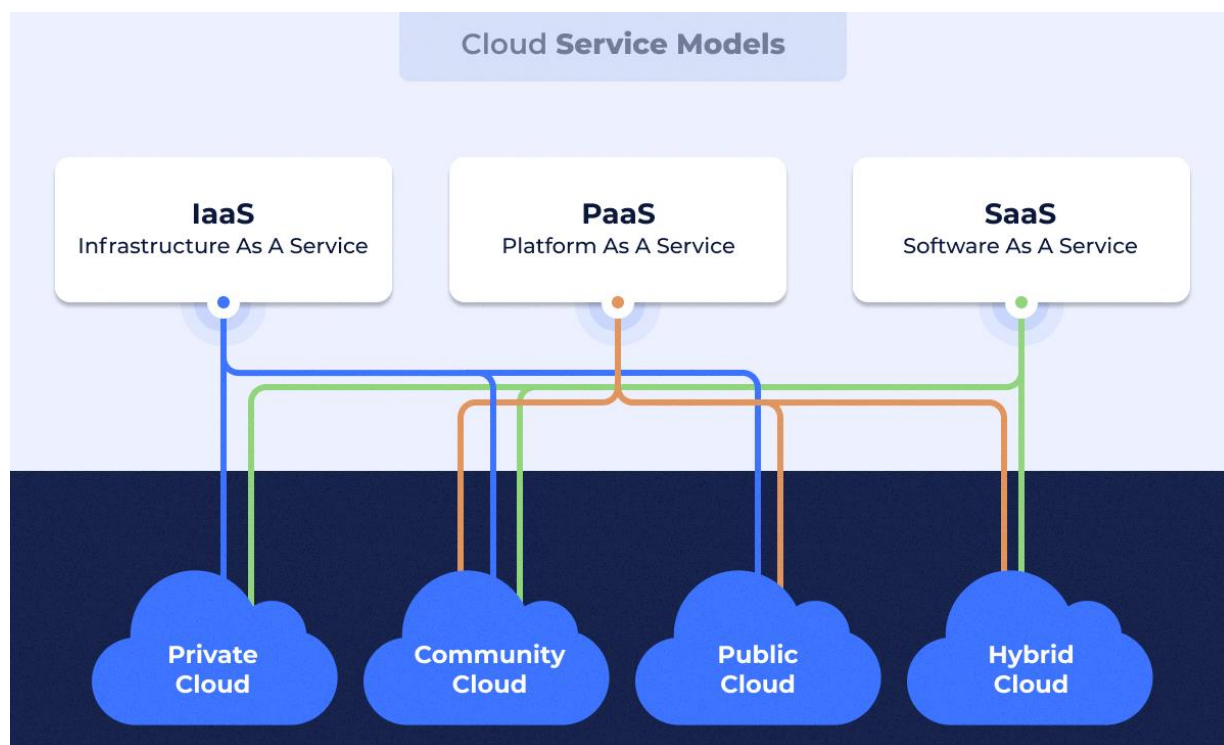
Disadvantages of SaaS

- . Less customization
- . Internet dependency
- . Subscription cost
- . Data privacy concern
- . Vendor dependency

- . Email
- . Online office tools

- . CRM
- . Video conferencing
- . Accounting software
- . Learning management systems

. 2.2 Deployment models:



Users access services through the internet.

Deployment model explains **where and how cloud infrastructure is deployed**.

Major deployment models are:

- . **Public cloud**
- . **Private cloud**
- . **Hybrid cloud**
- . **Community cloud**

A. Public Cloud

Meaning: Public cloud is cloud infrastructure owned and managed by a cloud service provider and shared among many customers.

Examples:

- . AWS
- . Microsoft Azure
- . Google Cloud
- . Oracle Cloud
- . IBM Cloud

Public Cloud Example

A startup hosts its e-commerce website on AWS.

It uses:

- . AWS EC2 for server
- . AWS S3 for image storage
- . AWS RDS for database
- . CloudFront for content delivery

The startup does not own the physical data center.

Advantages of Public Cloud

- . Low initial cost
 - . Highly scalable
 - . Easy to start
 - . Global availability
 - . Pay-as-you-use pricing
-

Disadvantages of Public Cloud

- . Less direct control
 - . Data privacy concern
 - . Compliance issues for sensitive data
 - . Internet dependency
 - . Vendor lock-in risk
-

Best Use Cases

- . Website hosting
 - . Startups
 - . Online applications
 - . Backup storage
 - . Testing and development
 - . Public-facing applications
-

B. Private Cloud

Meaning

Private cloud is cloud infrastructure used by only one organization.

It may be located:

- . Inside the organization's own data center
 - . In a hosted private cloud facility
-

Private Cloud Example

A bank builds its own private cloud to manage customer records, internal banking applications, and transaction systems.

The infrastructure is not shared with other companies.

Advantages of Private Cloud

- . More control
 - . Better privacy
 - . Better customization
 - . Suitable for sensitive data
 - . Easier compliance management
-

Disadvantages of Private Cloud

- . High initial cost
 - . Requires skilled IT team
 - . Maintenance responsibility
 - . Scaling may be expensive
-

Best Use Cases

- . Banking
- . Government systems
- . Military systems

- . Hospitals with sensitive records
- . Large enterprises with strict compliance

- . Universities
- . Government offices
- . Large businesses with mixed workloads

C. Hybrid Cloud

Meaning

Hybrid cloud combines private cloud and public cloud.

Some systems run in private cloud, and some systems run in public cloud.

Hybrid Cloud Example

A hospital stores patient records in a private cloud but uses public cloud for website, email, and appointment booking.

Sensitive patient records → Private cloud

Public website and email → Public cloud

Advantages of Hybrid Cloud

- . Balance between security and scalability
- . Sensitive data can stay private
- . Public cloud can handle extra workload
- . Flexible cost management
- . Good disaster recovery option

Disadvantages of Hybrid Cloud

- . Complex management
- . Requires strong integration
- . Security policy must be carefully designed
- . Network connectivity is important

Best Use Cases

- . Hospitals
- . Banks

D. Community Cloud

Meaning

Community cloud is shared by organizations with similar needs, policies, or goals.

It is used by a specific community, not by the general public.

Community Cloud Example

Several government departments share one cloud infrastructure for citizen services.

Another example:

Several universities share a research cloud for academic data and simulations.

Advantages of Community Cloud

- . Shared cost
- . Common security policy
- . Suitable for similar organizations
- . Better collaboration
- . Industry-specific compliance

Disadvantages of Community Cloud

- . Limited scalability compared to public cloud
- . Governance may be difficult
- . Cost is higher than public cloud
- . Shared responsibility can be complex

- . Government departments
- . Research institutions
- . Healthcare networks
- . Educational institutions
- . Banking associations

2.3 Comparison of Service Models

IaaS vs PaaS vs SaaS

Feature	IaaS	PaaS	SaaS
Full Form	Infrastructure as a Service	Platform as a Service	Software as a Service
Main Service	Virtual hardware	Development platform	Ready-made software
User Manages	OS, app, data	App and data	Mostly data and user access
Provider Manages	Hardware, virtualization	Hardware, OS, runtime	Almost everything
Control	High	Medium	Low
Technical Skill Needed	High	Medium	Low
Example	AWS EC2	Heroku	Gmail

Page 10 of 28

Feature	IaaS	PaaS	SaaS
User Type	System admin, DevOps	Developer	End user
Customization	High	Medium	Low
Deployment Speed	Medium	Fast	Very fast

Simple Analogy

Model	Analogy	Explanation
IaaS	Renting empty land	You build everything yourself
PaaS	Renting a ready kitchen	You cook your own food
SaaS	Eating at restaurant	You just use the final service

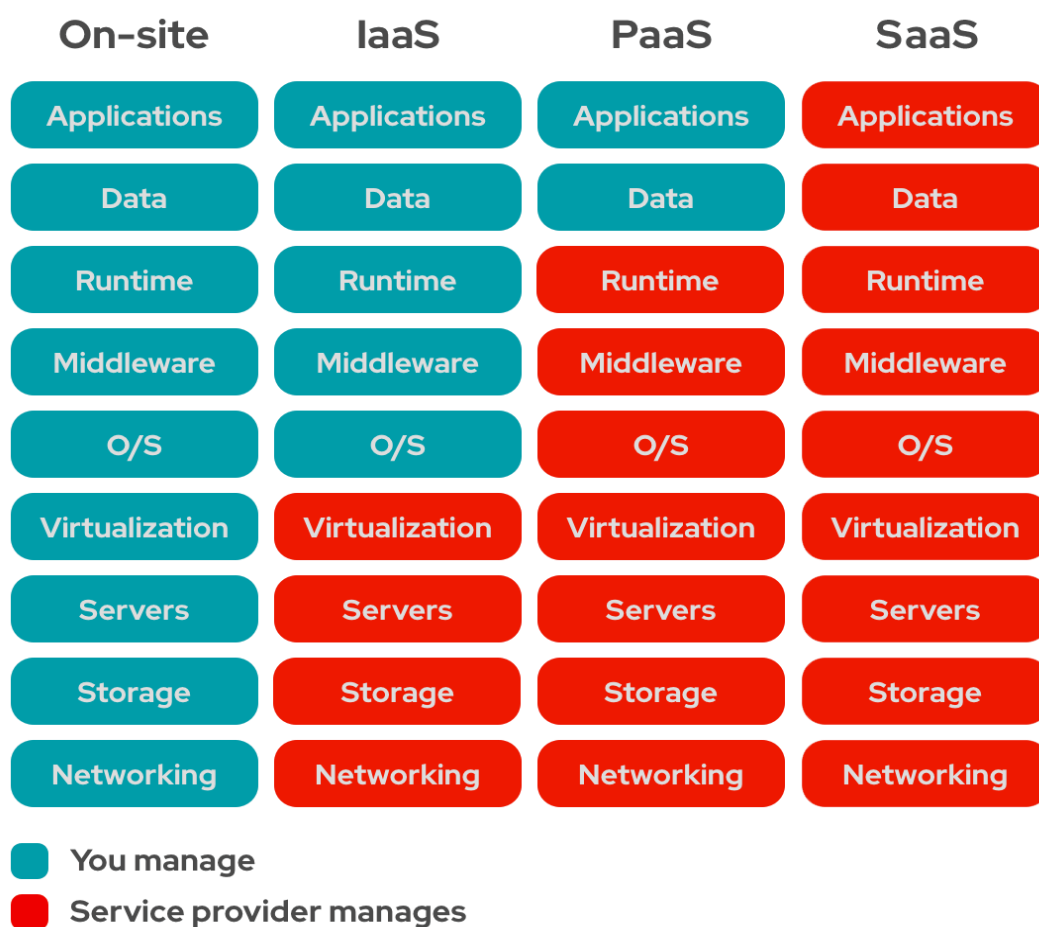
Responsibility Comparison

Layer	IaaS	PaaS	SaaS
Application	Customer	Customer	Provider
Data	Customer	Customer	Shared

Layer	IaaS	PaaS	SaaS
Runtime	Customer	Provider	Provider
Middleware	Customer	Provider	Provider
Operating System	Customer	Provider	Provider
Virtualization	Provider	Provider	Provider

Layer	IaaS	PaaS	Page 11 of 28 SaaS
Server Hardware	Provider	Provider	Provider
Network	Provider	Provider	Provider
Data Center	Provider	Provider	Provider

. 2.3 Comparison of service models



Feature	IaaS	PaaS	SaaS
Full Form	Infrastructure as a Service	Platform as a Service	Software as a Service
Main Service	Virtual hardware	Development platform	Ready-made software
User Manages	OS, app, data	App and data	Mostly data and user access
Provider Manages	Hardware, virtualization	Hardware, OS, runtime	Almost everything
Control	High	Medium	Low
Technical Skill Needed	High	Medium	Low
Example	AWS EC2	Heroku	Gmail
User Type	System admin, DevOps	Developer	End user
Customization	High	Medium	Low
Deployment Speed	Medium	Fast	Very fast

Simple Analogy

Model	Analogy	Explanation
IaaS	Renting empty land	You build everything yourself
PaaS	Renting a ready kitchen	You cook your own food
SaaS	Eating at restaurant	You just use the final service

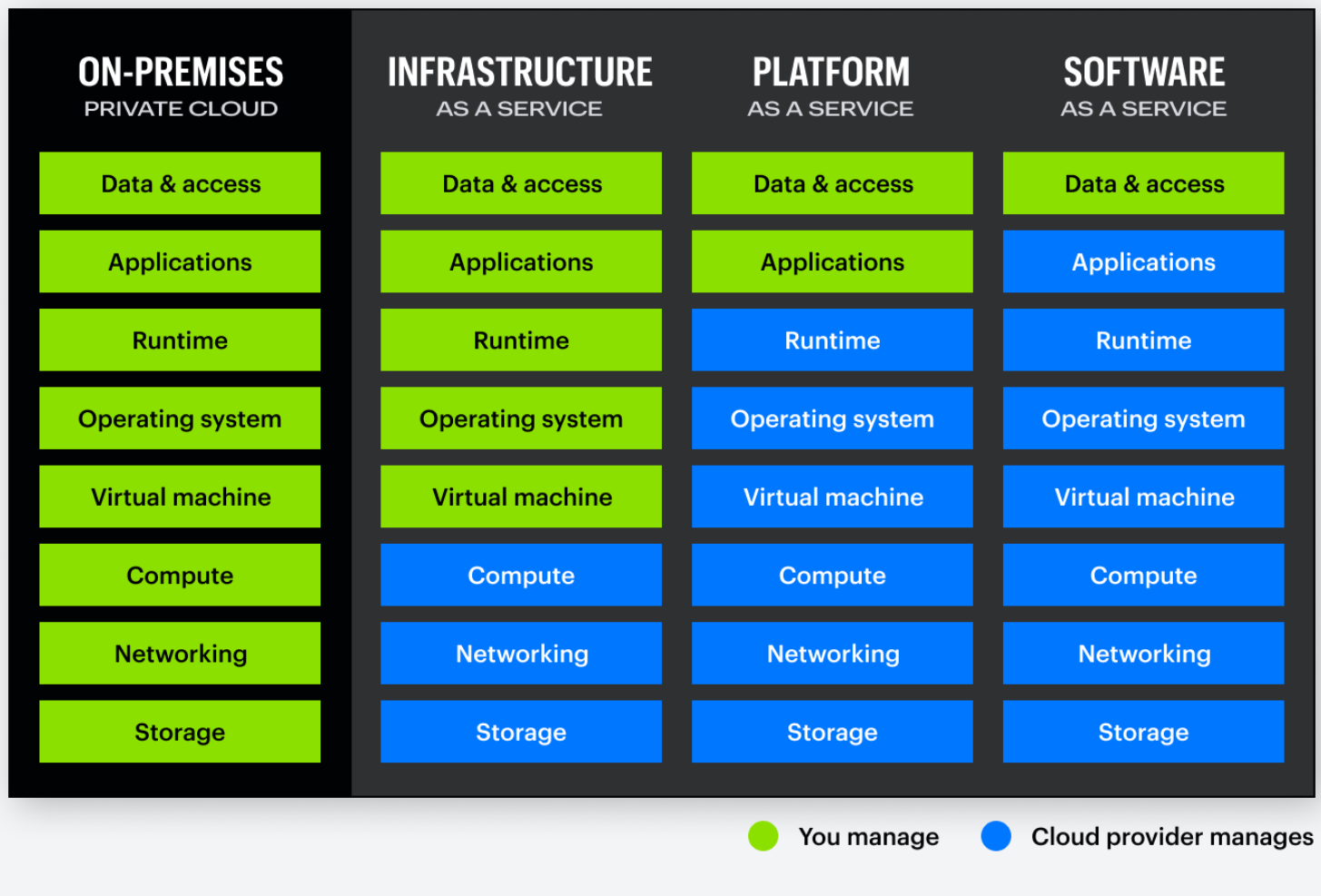
Responsibility Comparison

Layer	IaaS	PaaS	SaaS
Application	Customer	Customer	Provider
Data	Customer	Customer	Shared
Runtime	Customer	Provider	Provider
Middleware	Customer	Provider	Provider
Operating System	Customer	Provider	Provider
Virtualization	Provider	Provider	Provider
Server Hardware	Provider	Provider	Provider

Layer	IaaS	PaaS	SaaS
Network	Provider	Provider	Provider

Layer	IaaS	PaaS	Page 13 of 28 SaaS
Data Center	Provider	Provider	Provider

. 2.4 Shared responsibility model



Meaning

The shared responsibility model explains which security and management tasks are handled by the cloud provider and which are handled by the customer.

In cloud computing, security is not only the provider's responsibility. The customer also has responsibilities.

Basic Idea

Cloud Provider = Security OF the cloud
Customer = Security IN the cloud

Provider Responsibilities

The cloud provider is usually responsible for:

- . Physical data center security
- . Physical servers
- . Storage hardware
- . Network hardware
- . Power and cooling
- . Virtualization layer
- . Cloud infrastructure availability

Page 14 of 28

Responsibility	IaaS	PaaS	SaaS
Application	Customer	Customer	Provider
Data	Customer	Customer	Shared
User access	Customer	Customer	Customer
Password policy	Customer	Customer	Customer

Customer Responsibilities

The customer is usually responsible for:

- . User accounts
- . Passwords
- . Access permissions
- . Application security
- . Data security
- . Operating system patching in IaaS
- . Firewall rules
- . Backup configuration
- . Compliance settings

Example 1: IaaS Responsibility

A company rents an Ubuntu VM from a cloud provider.

Provider manages:

- . Physical server
- . Data center
- . Virtualization
- . Physical network

Customer manages:

- . Ubuntu updates
- . Nginx configuration
- . Database password
- . Firewall rules
- . Application security
- . User accounts

Shared Responsibility by Service Model

Responsibility	IaaS	PaaS	SaaS
Physical security	Provider	Provider	Provider
Network hardware	Provider	Provider	Provider
Virtualization	Provider	Provider	Provider
Operating system	Customer	Provider	Provider
Runtime	Customer	Provider	Provider

Example 2: SaaS Responsibility

A college uses Google Classroom.

Provider manages:

- . Application
- . Server
- . Storage system

- . Updates
- . Platform security

Common Security Mistake

College manages:

- . Teacher accounts
- . Student accounts
- . Passwords
- . Sharing permissions
- . Class access control

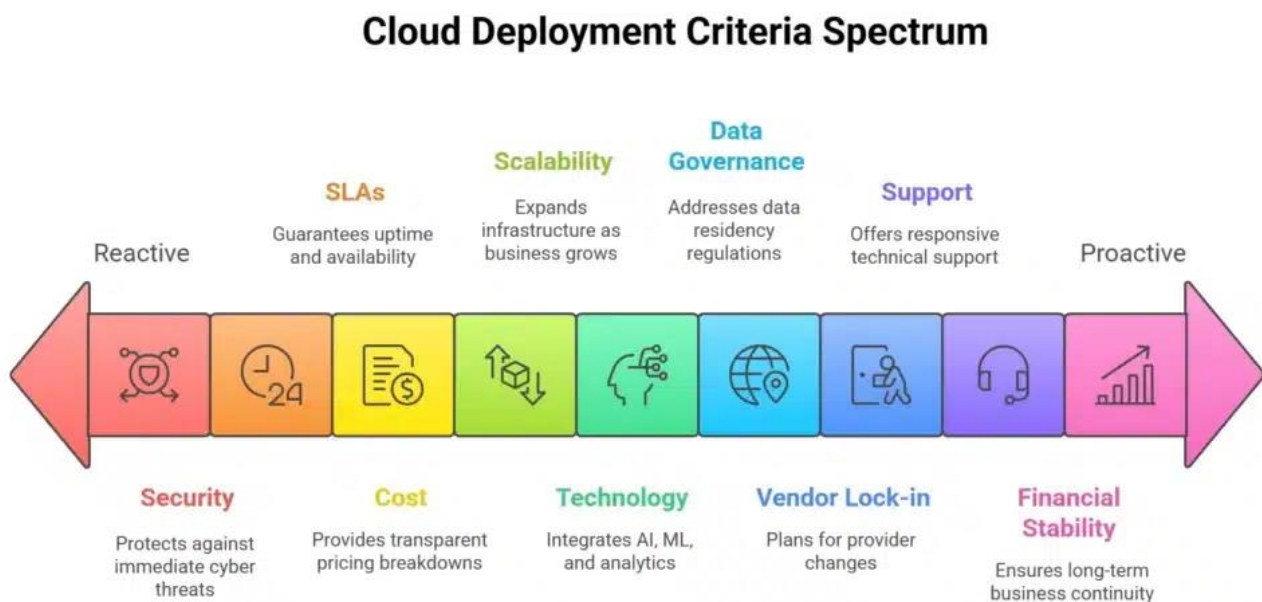
A user stores sensitive files in cloud storage and sets permission to:

Anyone with the link can view

Even if the cloud provider is secure, the data becomes exposed due to wrong customer configuration.

This shows why shared responsibility is important.

. 2.5 Selection criteria



Meaning

Selection criteria are the factors used to choose the correct cloud service model and deployment model.

- . Compliance
- . Performance
- . Customization
- . Data sensitivity
- . Migration complexity
- . Vendor support
- . Availability requirement

Main Selection Factors

- . Cost
- . Security
- . Control
- . Scalability
- . Technical skill

Selecting Service Model

Requirement	Suitable Model	Reason
Need full control over OS	IaaS	Customer manages server OS
Need to develop app quickly	PaaS	Platform handles runtime
Need ready-made email	SaaS	Complete software available
Need custom database tuning	IaaS	More control
Need simple office tools	SaaS	No technical setup
Need app deployment without server management	PaaS	Developer-focused

Selecting Deployment Model

Requirement	Suitable Model	Reason
Low cost and fast start	Public cloud	Easy and scalable

Requirement	Suitable Model	Reason
Sensitive data and strict control	Private cloud	More privacy
Mix of sensitive and public workloads	Hybrid cloud	Balanced model
Similar organizations sharing needs	Community cloud	Shared policy and cost

Scenario-Based Examples

Scenario 1: Small Startup Website

A startup wants to host an online shopping website with limited budget.

Best choice:

Service Model: IaaS or PaaS
Deployment Model: Public Cloud

Reason:

- . Low initial cost
- . Easy scaling
- . No physical server required
- . Fast deployment

Scenario 2: Hospital Patient Record System

A hospital wants to store patient records securely.

Best choice:

Service Model: IaaS or Private SaaS
Deployment Model: Private Cloud or Hybrid Cloud

Reason:

- . Sensitive patient data
- . Privacy requirement
- . Controlled access needed
- . Backup and disaster recovery important

Scenario 3: College Email and Document System

A college wants email, document sharing, and online class tools.

Best choice:

Service Model: SaaS
Deployment Model: Public Cloud

Reason:

- . Ready-made software
- . Easy to use
- . No server maintenance
- . Supports collaboration

Scenario 4: Software Company Building API

A software company wants to develop and deploy REST API quickly.

Best choice:

Service Model: PaaS or Container Platform
Deployment Model: Public Cloud

Reason:

- . Developer can focus on code
- . Platform manages runtime
- . Easy deployment
- . Auto scaling possible

Scenario 5: Government Citizen Data System

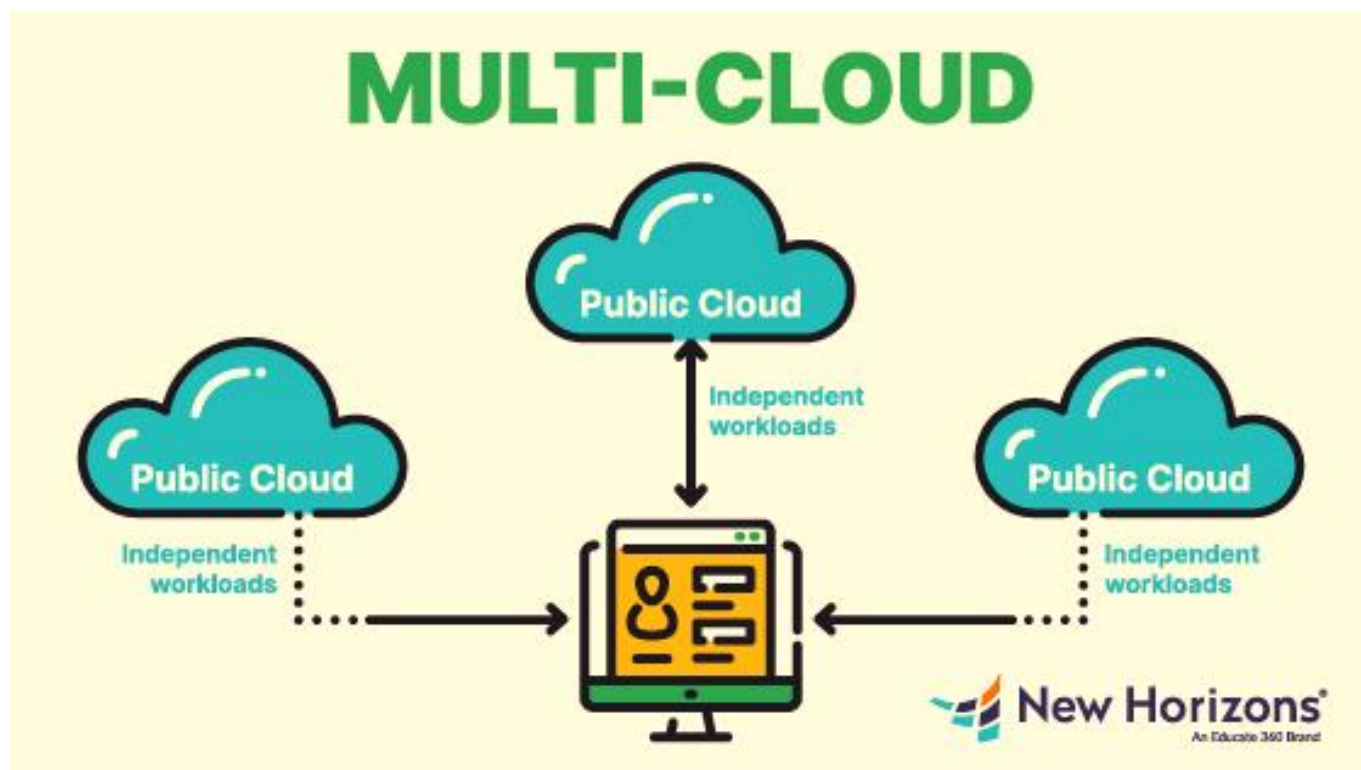
A government department wants a secure citizen database.

Best choice:

Service Model: IaaS or Private Cloud Platform
Deployment Model: Private Cloud or Community Cloud

Reason:

- . Sensitive data
- . Compliance requirement
- . More control required
- . Shared government infrastructure possible



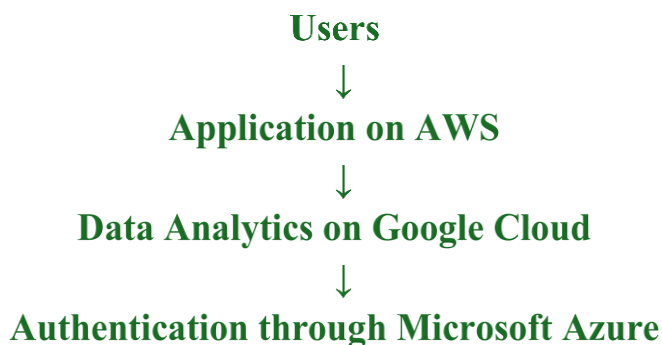
Meaning of Multi-Cloud

Multi-cloud means using cloud services from more than one cloud provider.

Example:

*A company uses AWS for web hosting,
Google Cloud for data analytics,
and Microsoft Azure for office identity services.*

Multi-Cloud Architecture Example



Why Organizations Use Multi-Cloud

- . *Avoid dependency on one provider*
 - . *Use best service from each provider*
 - . *Improve availability*
 - . *Reduce outage risk*
 - . *Meet regional or legal requirements*
 - . *Optimize cost*
 - . *Improve disaster recovery*
-

Multi-Cloud Example in Business

A company uses:

Workload	Cloud Provider
Website hosting	AWS

Workload	Cloud Provider
Office email	Microsoft 365
Data analytics	Google Cloud
Backup storage	Wasabi / Backblaze
Video meeting	Zoom

This is a multi-cloud environment.

Advantages of Multi-Cloud

- . *Reduces vendor dependency*
 - . *Improves flexibility*
 - . *Allows best-of-breed services*
 - . *Better disaster recovery*
 - . *Can improve negotiation power*
-

Challenges of Multi-Cloud

- . *Complex management*
 - . *Different billing systems*
 - . *Different security policies*
 - . *Skill requirement increases*
 - . *Monitoring becomes harder*
 - . *Data transfer cost may increase*
-

Data Example: Multi-Cloud Cost

A company uses two cloud providers.

Service	Provider	Monthly Cost
Web hosting	AWS	Rs. 20,000
Email	Microsoft 365	Rs. 15,000
Analytics	Google Cloud	Rs. 12,000

Service	Provider	Monthly Cost
Backup	Backblaze	Rs. 5,000

Total monthly cost:

$$20,000 + 15,000 + 12,000 + 5,000 = \text{Rs. } 52,000$$

Yearly cost:

$$52,000 \times 12 = \text{Rs. } 624,000$$

. 2.7 Vendor lock-in issues



Meaning of Vendor Lock-In

Vendor lock-in means becoming highly dependent on one cloud provider, making it difficult, costly, or risky to move to another provider.

Simple Example

A company builds its application using many provider-specific services.

Example:

- . *AWS Lambda*
- . *AWS DynamoDB*
- . *AWS S3*
- . *AWS API Gateway*

Later, if the company wants to move to Azure or Google Cloud, it must redesign many parts of the application.

Causes of Vendor Lock-In

- . *Provider-specific APIs*
- . *Proprietary databases*
- . *Special serverless functions*

Sanjeev Thapa, Er. DevOps, SRE, CKA, RHCSA, RHCE, RHCSA-Openstack, MTCNA, MTCTCE, UBSRS, HEv6, Research Evangelist

- . *Specific storage format*
- . *Licensing restrictions*
- . *Data transfer cost*
- . *Lack of documentation*
- . *Application tightly connected with one provider*

Vendor Lock-In Example

A company uses a cloud database that is only available in one provider.

Problem Area	Effect
Database syntax	May not work in another cloud
API dependency	Code must be rewritten
Data migration	Takes time and cost
Staff skill	Team must learn new platform
Downtime risk	Migration may interrupt service

Data-Based Example: Migration Cost



A company wants to move from **Cloud A** to **Cloud B**.

Migration Activity	Estimated Cost
Data export	Rs. 50,000
Developer code changes	Rs. 300,000
Testing	Rs. 100,000
Training	Rs. 80,000
Downtime loss	Rs. 150,000

Total migration cost:

$$50,000 + 300,000 + 100,000 + 80,000 + 150,000 \\ = \text{Rs. } 680,000$$

This high cost is an example of vendor lock-in impact.

How to Reduce Vendor Lock-In

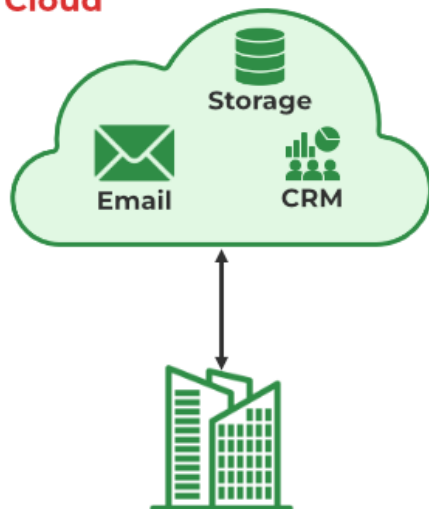
- . *Use open standards.*
- . *Use containers such as Docker.*
- . *Use Kubernetes.*
- . *Use standard databases like PostgreSQL or MySQL.*
- . *Avoid unnecessary provider-specific services.*
- . *Keep backup copies in standard formats.*
- . *Use Infrastructure as Code.*
- . *Document architecture properly.*
- . *Design application with portability in mind.*

Vendor Lock-In vs Multi-Cloud

Topic	Vendor Lock-In	Multi-Cloud
Meaning	Dependency on one provider	Use of multiple providers
Risk	Difficult migration	Complex management

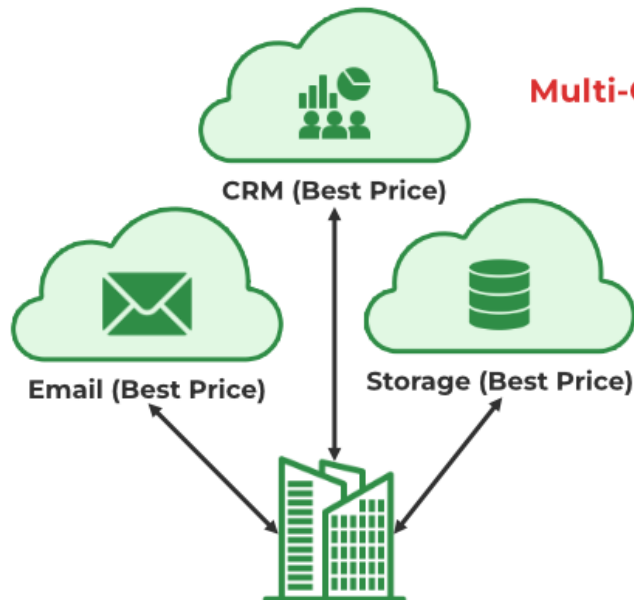
Topic	Vendor Lock-In	Multi-Cloud
Main Problem	Lack of flexibility	Operational complexity
Solution	Open standards, containers	Unified monitoring and governance

Single Cloud



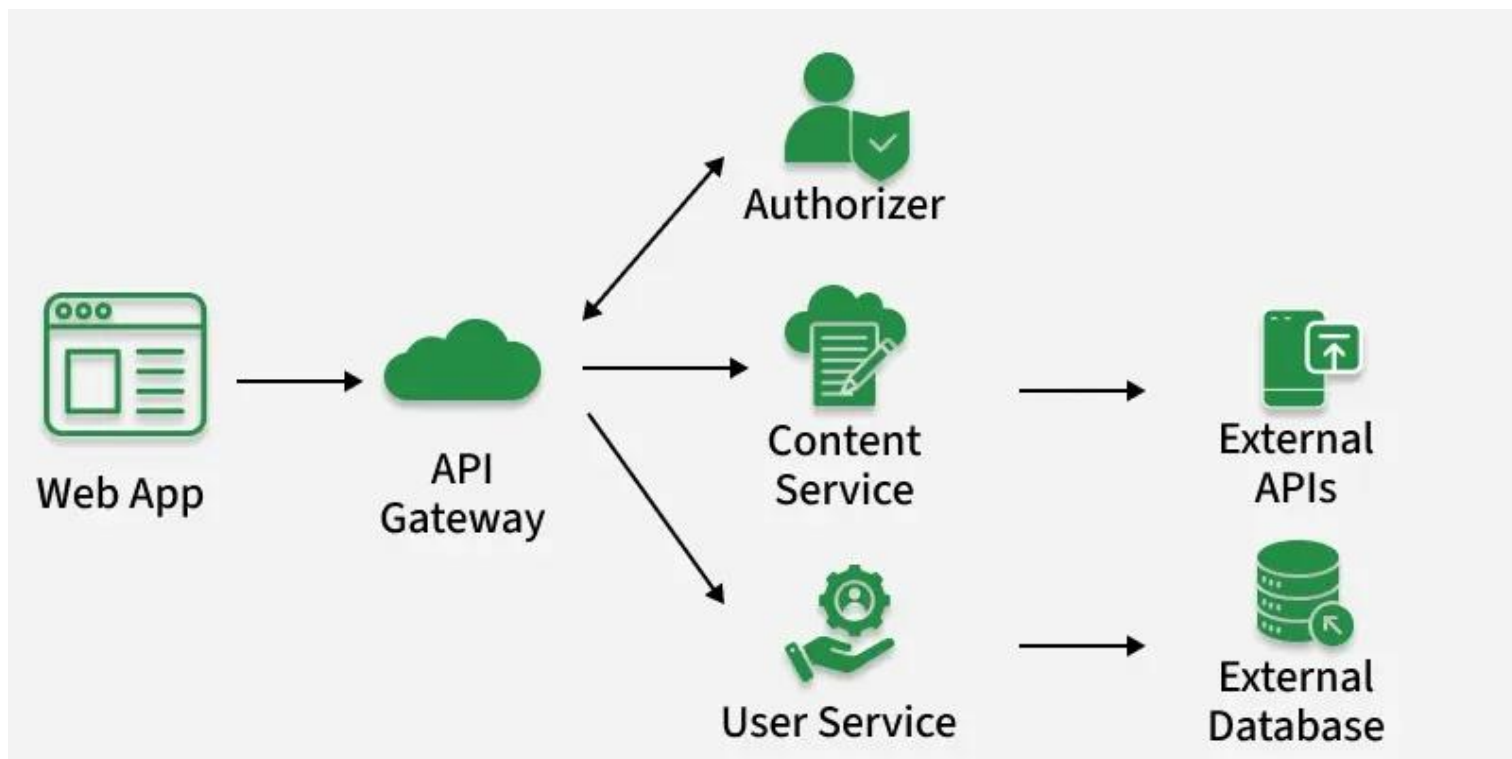
Company is Locked into One Cloud Vendor

Multi-Cloud



Company has More Flexibility with Multiple Cloud Vendors

. 2.8 Serverless computing and event-driven architecture



Serverless Computing

Meaning

Serverless computing is a cloud model where developers run code without managing servers.

The cloud provider automatically manages:

- . *Server provisioning*
- . *Runtime*
- . *Scaling*
- . *Availability*
- . *Infrastructure maintenance*

Serverless does not mean there is no server. It means the customer does not manage the server.

Serverless Examples

- . *AWS Lambda*
- . *Azure Functions*

Sanjeev Thapa, Er. DevOps, SRE, CKA, RHCSA, RHCE, RHCSA-Openstack, MTCNA, MTCTCE, UBSRS, HEv6, Research Evangelist

- . *Google Cloud Functions*
- . *Cloudflare Workers*
- . *Vercel Functions*

Serverless Example Scenario

A college has an online admission form.

When a student submits the form, a function automatically:

- . *Validates the form*
- . *Saves data to database*
- . *Sends confirmation email*
- . *Sends notification to admission officer*

The function runs only when a form is submitted.

Serverless Benefits

- . No server management
- . Auto scaling
- . Pay only when function runs
- . Good for event-based tasks
- . Faster development

Serverless Challenges

- . Cold start delay
- . Limited execution time
- . Vendor lock-in risk
- . Debugging can be difficult
- . Not suitable for long-running applications

Event-Driven Architecture

Meaning

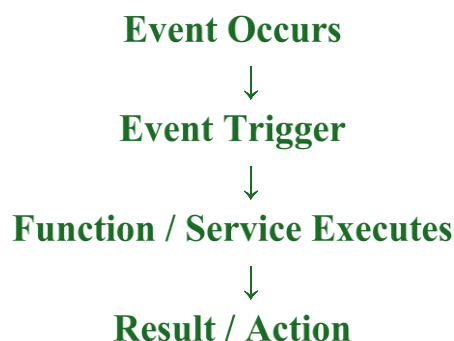
Event-driven architecture is a design model where system components communicate by producing and responding to events.

An event is an action or change in the system.

Examples of events:

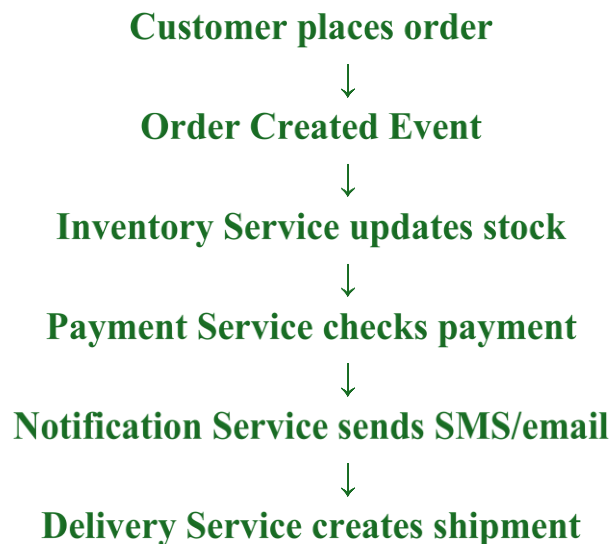
- . User registered
- . Payment completed
- . File uploaded
- . Order placed
- . Sensor data received
- . Database record updated

Event-Driven Flow



Sanjeev Thapa, Er. DevOps, SRE, CKA, RHCSA, RHCE, RHCSA-Openstack, MTCNA, MTCTCE, UBSRS, HEv6, Research Evangelist

Example: Online Shopping Event Flow



Event-Driven Architecture Components

Component	Meaning
Event Producer	Creates event
Event	Action/change information
Event Broker	Transfers event
Event Consumer	Receives and processes event
Function	Code executed after event

A college receives admission forms.

Time	Forms Submitted
9 AM	10
10 AM	25
11 AM	40
12 PM	75
1 PM	30

Each form triggers one serverless function.

Total function executions:

$$10 + 25 + 40 + 75 + 30 = 180 \text{ executions}$$

If cost per 1,000 executions is Rs. 20:

$$\text{Cost} = (180 \div 1000) \times 20$$

$$\text{Cost} = \text{Rs. } 3.60$$

This shows serverless can be cost-effective for event-based workloads.

Simple Python Event-Driven Example

```
def admission_event_handler(event):  
    student_name = event["student_name"]  
    program = event["program"]  
  
    message = f"Application received from  
{student_name} for {program}."  
    return message
```

```
        "student_name": "Ram Thapa",  
        "program": "BSc CSIT"  
    }
```

```
result = admission_event_handler(event_data)  
print(result)
```

Output:

Application received from Ram Thapa for BSc CSIT.

Serverless vs Traditional Server

Feature	Traditional Server	Serverless
Server management	Customer manages	Provider manages
Scaling	Manual or configured	Automatic
Cost	Pay while server runs	Pay per execution
Best for	Long-running apps	Event-based tasks
Startup	Always running	May have cold start
Control	More control	Less control

Use serverless for:

- . *Form submission processing*
- . *Image resizing*
- . *Email notification*
- . *Scheduled jobs*
- . *Payment confirmation*
- . *Log processing*
- . *IoT sensor processing*

When Not to Use Serverless

Avoid serverless when:

- . *Application must run continuously*
- . *Long processing time is required*
- . *Full server control is needed*
- . *Very low-latency response is required*
- . *Heavy custom runtime is required*

Practical Commands and Small Programs

1. Run a Simple Web Server on IaaS VM

Assume Ubuntu VM is rented from cloud.

Install Nginx:

```
sudo apt update
sudo apt install nginx -y
sudo systemctl start nginx
sudo systemctl enable nginx
```

Check status:

```
sudo systemctl status nginx
```

Test locally:

```
curl http://localhost
```

Create folder:

```
mkdir cloud-service-demo
cd cloud-service-demo
```

Create index.html:

```
nano index.html
```

Paste:

```
<!DOCTYPE html>
<html>
<head>
  <title>Cloud Service Model Demo</title>
</head>
<body>
  <h1>Hello Cloud Service Models</h1>
  <p>This application is running inside a
  Docker container.</p>
</body>
</html>
```

Create Dockerfile:

```
nano Dockerfile
```

Paste:

```
FROM nginx:latest
COPY index.html
/usr/share/nginx/html/index.html
```

Build image:

```
docker build -t cloud-service-demo .
```

Run container:

```
docker run -d -p 8080:80 --name service-demo
cloud-service-demo
```

Open:

```
http://localhost:8080
```

Stop:

3. Simple Event-Driven Program

```
def payment_event(event):  
    customer = event["customer"]  
    amount = event["amount"]  
    status = event["status"]  
  
    if status == "success":  
        return f"Payment of Rs. {amount}  
received from {customer}. Send receipt."  
    else:  
        return f"Payment failed for {customer}.  
Send retry message."
```

```
event = {  
    "customer": "Sita Sharma",  
    "amount": 2500,  
    "status": "success"  
}  
  
print(payment_event(event))
```

Output:

*Payment of Rs. 2500 received from Sita
Sharma. Send receipt.*

PAPER

<https://www.grycap.upv.es/gmolto/publications/preprints/Perez2019osc.pdf>

On-premises Serverless Computing for Event-Driven Data Processing Applications

Alfonso Pérez, Sebastián Risco, Diana María Naranjo, Miguel Caballer, Germán Moltó
Instituto de Instrumentación para Imagen Molecular (I3M).
Centro mixto CSIC - Universitat Politècnica de València
Camino de Vera s/n, 46022 Valencia, España
Email: alpegon3@upv.es, gmolto@dsic.upv.es, {serisgal,dnaranjo,micafer1}@i3m.upv.es

Abstract—The advent of open-source serverless computing frameworks has introduced the ability to bring the Functions-as-a-Service (FaaS) paradigm for applications to be executed on-premises. In particular, data-driven scientific applications can benefit from these frameworks with the ability to trigger scalable computation in response to incoming workloads of files to be processed. This paper introduces an open-source framework to achieve on-premises serverless computing for event-driven data processing applications that features: i) the automated provisioning of an elastic Kubernetes cluster that can grow and shrink, in terms of the number of nodes, on multi-Clouds; ii) the automated deployment of a FaaS framework together with a data storage back-end that triggers events upon file uploads; iii) a service that provides a REST API to orchestrate the creation of such functions and iv) a graphical user interface that provides a unified entry point to interact with the aforementioned services. Together, this provides a framework to deploy a computing platform to create highly-parallel event-driven file-processing serverless applications that execute on customized runtime environments provided by Docker containers that run on an elastic Kubernetes

a customized execution environment provided by a Linux container. Although sometimes used interchangeably, FaaS typically represents a subset of Serverless computing, a trend based on creating application architectures that entirely rely on Cloud services that provide automated resource provisioning on behalf of (and transparent to) the user. Thus, by not explicitly managing servers, developers can focus on the definition of the application logic instead of devoting time to infrastructure provision, configuration and scalability. The SPEC Cloud Group [1] defines three key features of serverless cloud architectures: i) granular billing; the user is only charged when the application is running; ii) minimal operational logic: the Cloud provider is responsible for resource management and autoscaling and iii) event-driven: short-lived execution of functions in response to events.

Public Cloud providers include in their portfolios services to support FaaS. As an example, Amazon Web Services (AWS)

5 short presentation topics for this paper:

1. Serverless Computing and FaaS Basics

Explain what serverless computing is and how **Function-as-a-Service** works.

2. Problem with Public Cloud Serverless

Discuss limits of AWS Lambda-like platforms, such as execution time, storage limits, and less control.

3. OSCAR Framework

Introduce **OSCAR** as an open-source, on-premises serverless framework for data-processing applications.

4. OSCAR Architecture and Workflow

Explain how **Kubernetes, OpenFaaS, MinIO, Kaniko, and CLUES** work together to process files automatically.

5. Case Study and Results

Explain the video object-detection experiment and how OSCAR handled workloads using elastic scaling.